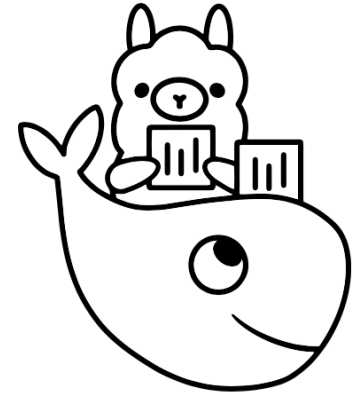


# CSE 322

Generative AI powered by Docker

# Ollama



- Ollama is an open-source tool to run large language models (LLMs) **locally**, providing a private and efficient alternative to cloud-based AI services.
- Ollama has a command-line interface (CLI) in order to download & run open-source LLMs **locally**.

```
$ docker pull ollama/ollama
```

GitHub repo: <https://github.com/ollama/ollama>

# Boot up LLM manager

- Run ollama in CPU only mode as follows:

```
$ docker run -d -v ollama:/root/.ollama -p \
    11434:11434 --name ollama ollama/ollama
```

More options: <https://hub.docker.com/r/ollama/ollama>

# Manager commands

```
$ docker exec -it ollama ollama help
```

## Available Commands:

|        |                                 |
|--------|---------------------------------|
| serve  | Start ollama                    |
| create | Create a model from a Modelfile |
| show   | Show information for a model    |
| run    | Run a model                     |
| stop   | Stop a running model            |
| pull   | Pull a model from a registry    |
| push   | Push a model to a registry      |
| list   | List models                     |
| ps     | List running models             |
| cp     | Copy a model                    |
| rm     | Remove a model                  |
| help   | Help about any command          |

# Download & deploy an LLM

- Download and run a specific LLM as follows:

```
# llama3
```

```
$ docker exec -it ollama ollama run llama3
```

```
# mistral
```

```
$ docker exec -it ollama ollama run mistral
```

```
# deepseek r1
```

```
$ docker exec -it ollama ollama run deepseek-r1
```

For more models: <https://ollama.com/search>

# System requirements

- You should have **at least**:  
8 GB of RAM available to run the 7B models,  
16 GB to run the 13B models, and  
32 GB to run the 33B models.

# Ollama Python Library

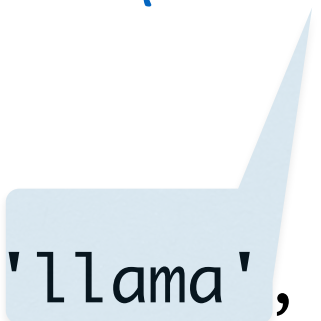
- The Ollama Python library provides the easiest way to integrate Python 3.8+ projects with [Ollama](#).
- When an LLMA model is already pulled and running on ollama; then, you can give instructions to your model through the Python library.
- First integrate the package into your Python project as:

```
$ pip install ollama
```

GitHub repo: <https://github.com/ollama/ollama-python>

# Ollama Python Library

LLM model pulled & running



```
from ollama import chat
from ollama import ChatResponse

response: ChatResponse = chat(model='llama',
messages=[
    {
        'role': 'user',
        'content': 'What is the capital of Germany ?',
    },
])
print(response.message.content)
```

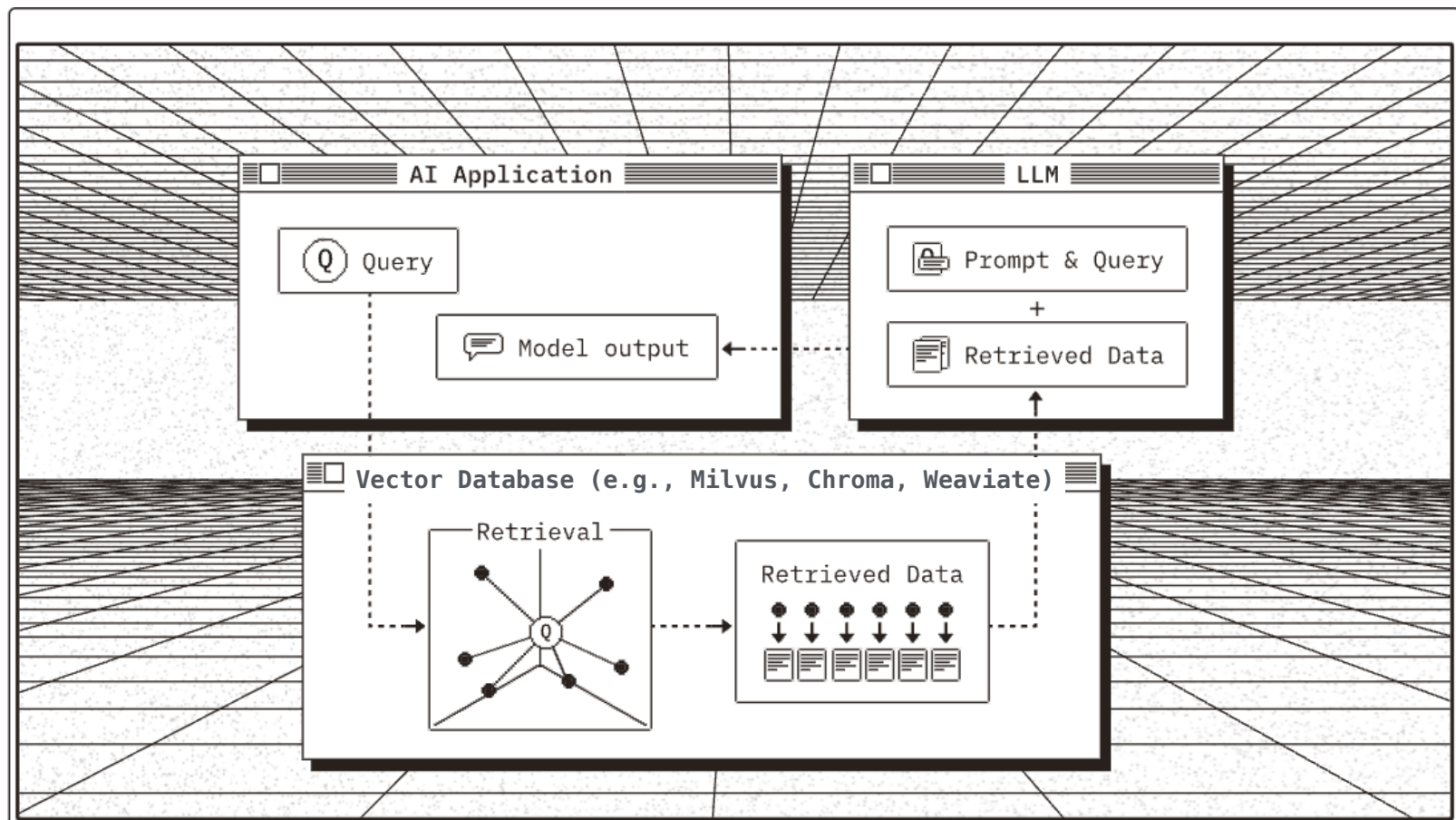




# Augmented Generation

- Retrieval Augmented Generation (RAG)  
<https://github.com/aishwaryanr/awesome-generative-ai-guide>  
<https://github.com/Danielskry/Awesome-RAG>
- Table Augmented Generation (TAG)  
<https://github.com/TAG-Research/TAG-Bench>
- <Your Idea> Augmented Generation (YAG)  
**THE SKY IS THE LIMIT; BRAIN IT ON !**

# Retrieval Augmented Generation (RAG)



# Some Vector Databases

- **Milvus** is *open source*; ideal for large-scale applications.
- **Qdrant** comes with Docker-based setup.
- **Weaviate** is *open source*; Docker-compose configs.
- **Chroma** is a lightweight, Python-based *embedding* database.
- **Vectra** is for Node.js; stores indexes in *local file system*.

Visit: <https://dev.to/mehmetakar/local-vector-databases-1k8i>

# Smoke Test with Milvus Lite

Step 1. Install PyTorch, PyMilvus (and its model):

```
$ python3.12 -m venv venv && source venv/bin/activate  
$ (venv) pip install -U pymilvus  
$ (venv) pip install -U "pymilvus[model]"  
$ (venv) pip install -U torch
```

Step 2. Set up a **Vector DB**.

Step 3. Create a collection (a *relation* in the RDBMS lingo).

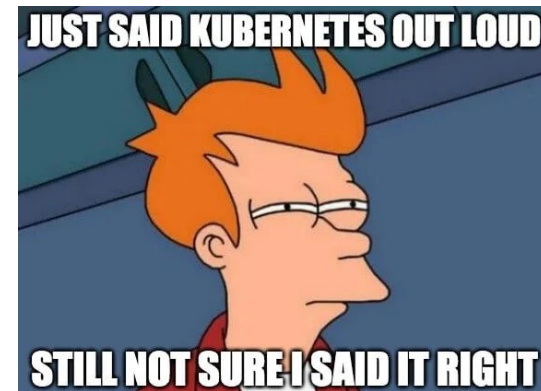
Step 4. **Embed** your custom dataset —> **Vectorization**.

Step 5. Do **Vector** “*semantic*” **Search** on your collection.

For more information: <https://milvus.io/docs/quickstart.md>

# Milvus Lite

- Python package *pymilvus* contains both the python client library and Milvus Lite, which can be embedded into the client application, i.e. your web application (think of it as *sqlite3* in the RDBBS lingo).
- Note that Milvus supports deployment on Docker as well as Kubernetes for production use cases.



# Container runtime & orchestration lingo

- *kubernetes* is pronounced as **coo-ber-net-ees**.
- *kubectrl* is pronounced **coob-control**.
- *etcd* is pronounced **etsy d**.
- *kubelet* is pronounced **coob-let**.



# Redis as Vector DB

- You can use Redis to set up a vector database.
- The steps are roughly similar as in the Milvus setup.
  - Create a Redis vector database
  - Create vector embeddings and store vectors
  - Query data and perform a vector search

See: <https://redis.io/docs/latest/develop/get-started/vector-database/>